



Advanced Computer Architecture

Lab1: Vector Processor Design Space Exploration

J. Parker Jones, Daniel Mueller-Gritschneider

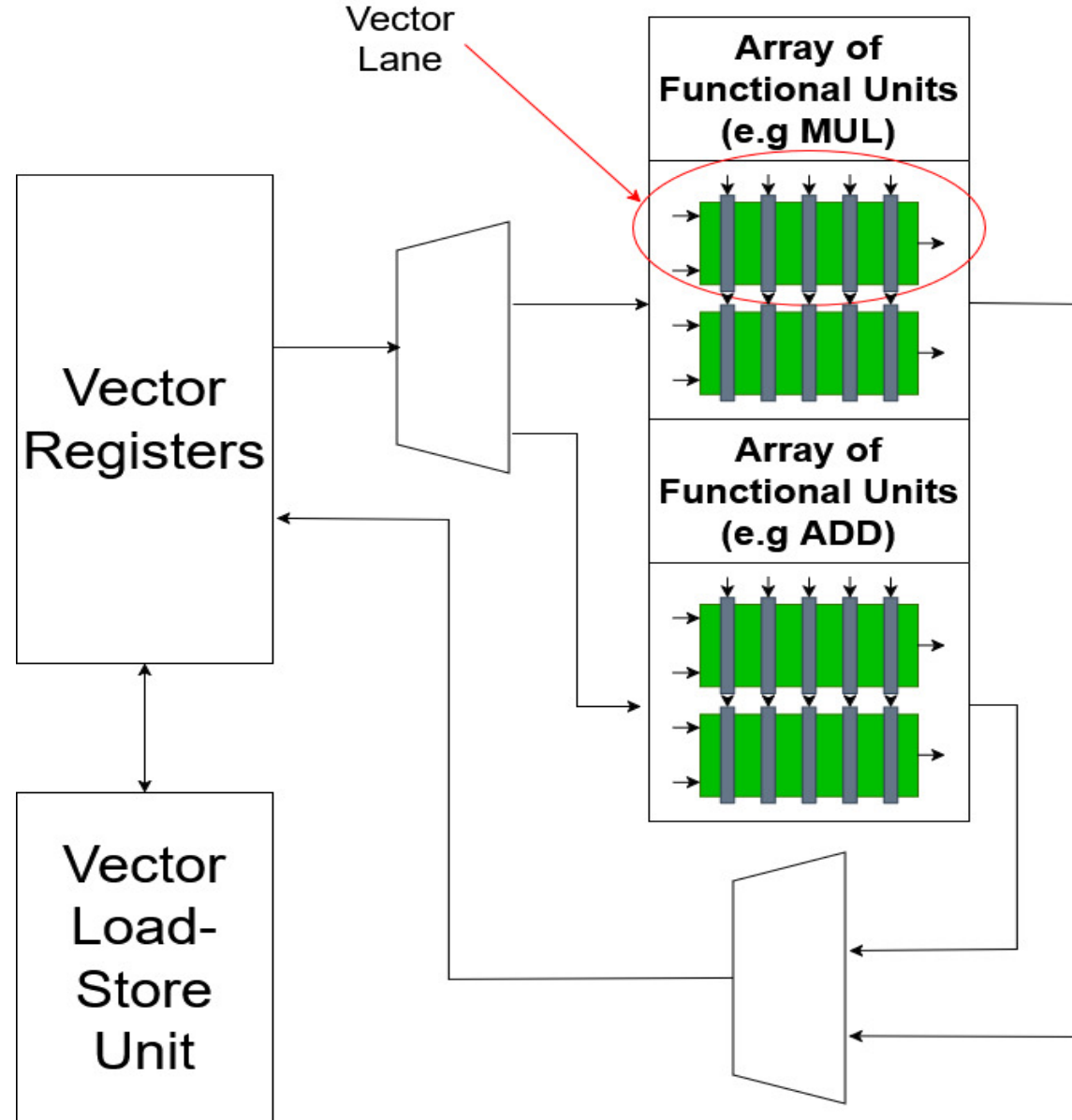
30.10.2025

Recap - Vector Units and RISC-V Vector Extension

Vector Instruction Sets

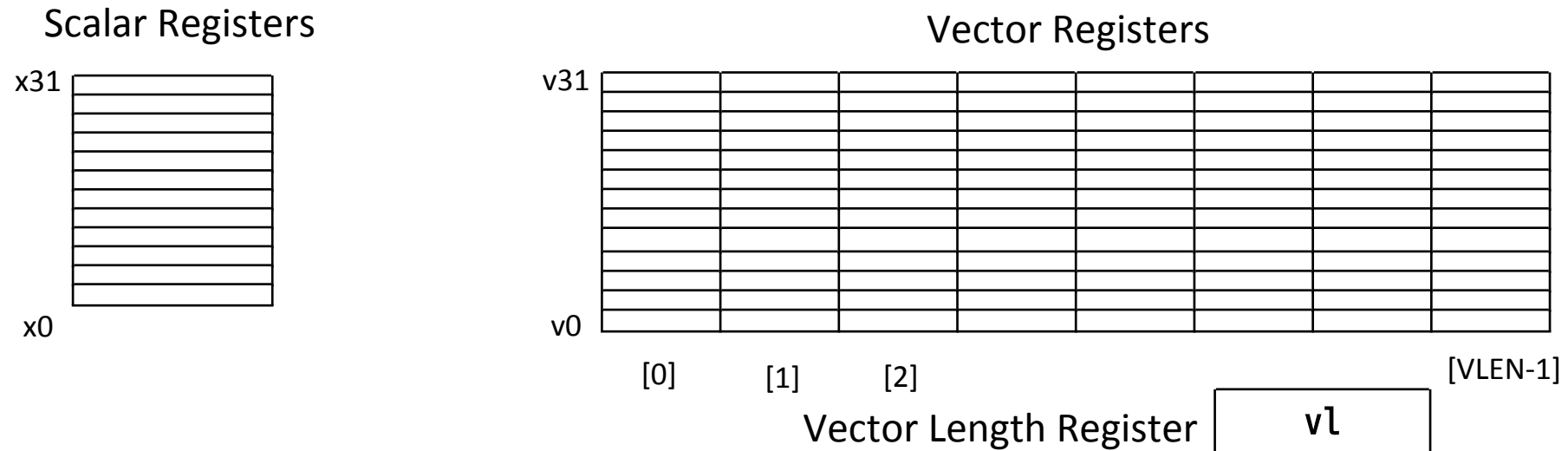
- One instruction operates on several data values (SIMD)
- The data values are independent
- Operation use the same type of functional unit for all data
- Data values are stored in separate registers
- Data values are arranged in uniform structure (vector)
- Load/Stores access
 - a continuous range of memory
 - use a regular pattern (strided load/store)
- One instruction stream for parallel pipelines (so called lanes)

Basic Structure of a Vector Unit



RISC-V Vector Programming Model

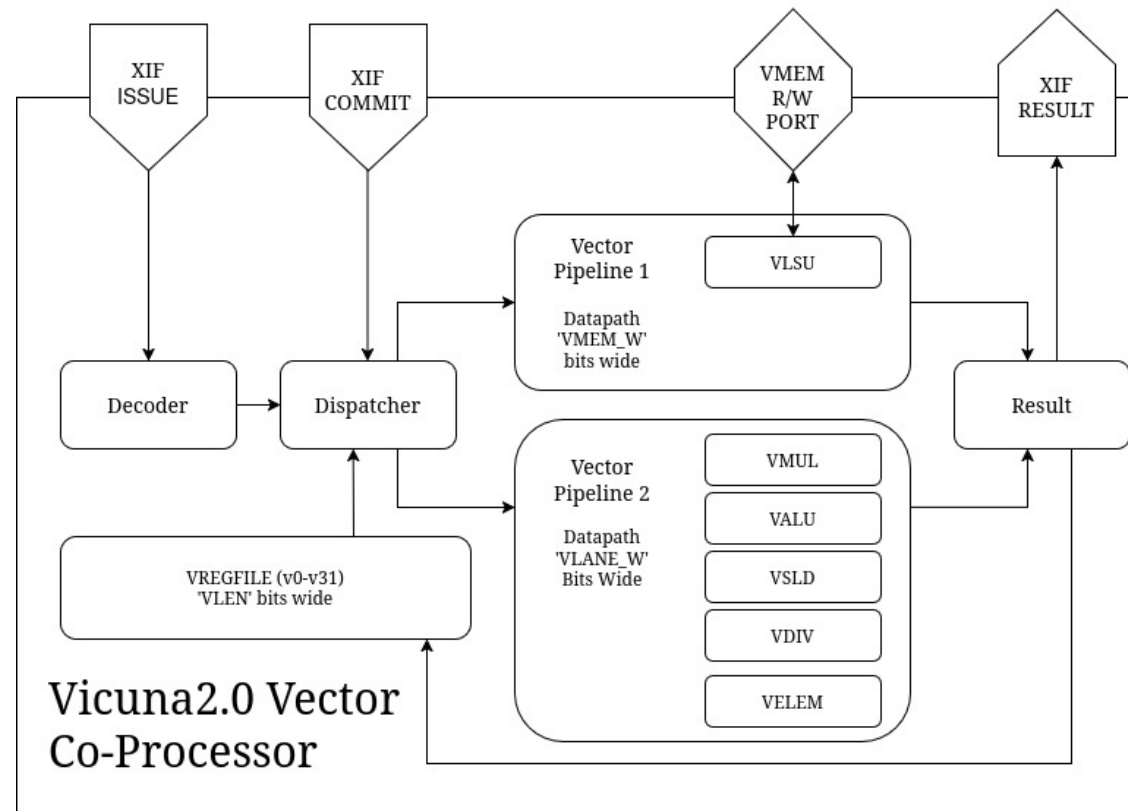
- Vector Registers and vector length
 - 32 vector data registers (v0~v31) : each **VLEN** bits long
 - Vector length register **vl**
 - defines on how many elements will the next vector operations be executed
 - Vector type register **vtype**
 - Contains **sew** and **lmul**



Vicuna2.0

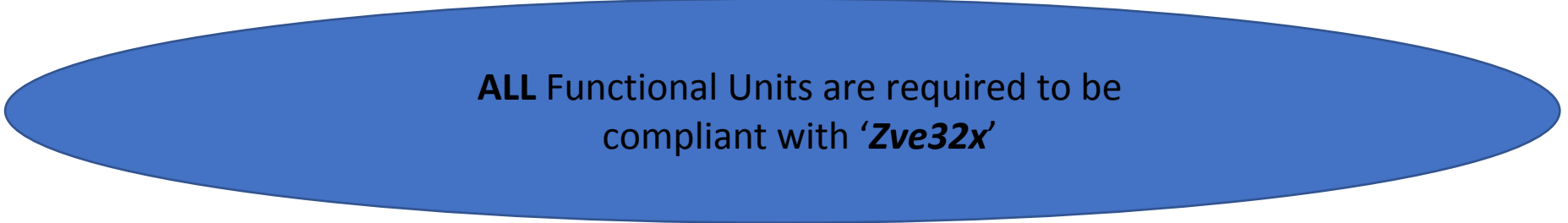
Vicuna2.0 Vector Co-Processor

- Open-Source, developed and maintained by TU Wien
- Targets the embedded subset of the RISC-V Vector specification (Zve32x, Zve32f)



Vicuna2.0 : Vector Functional Units

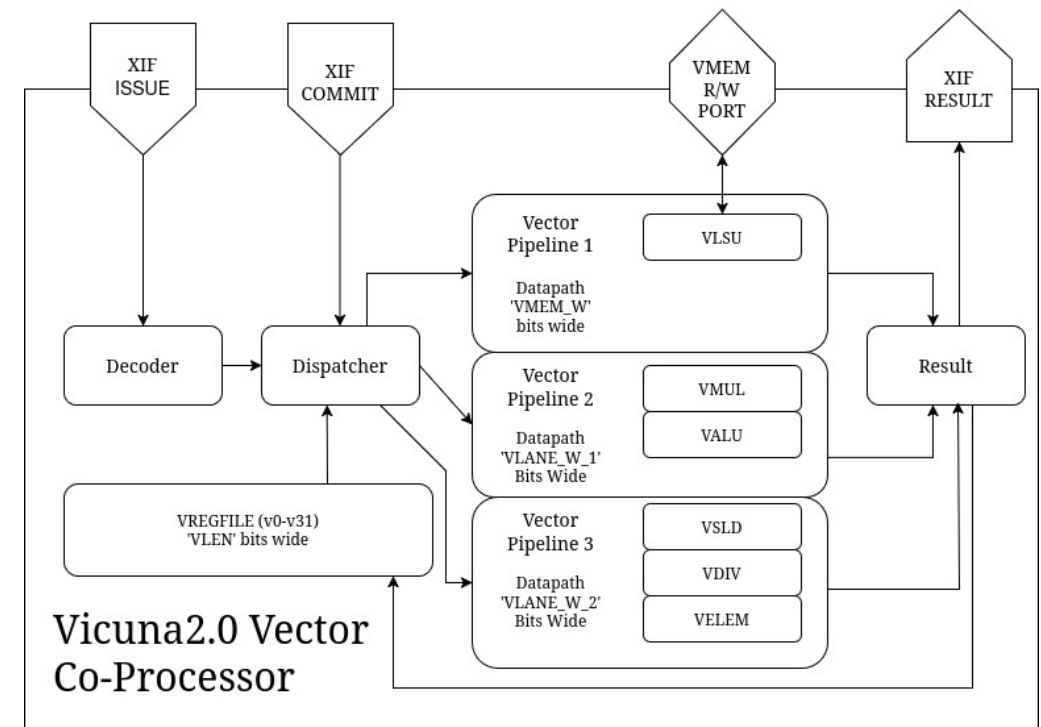
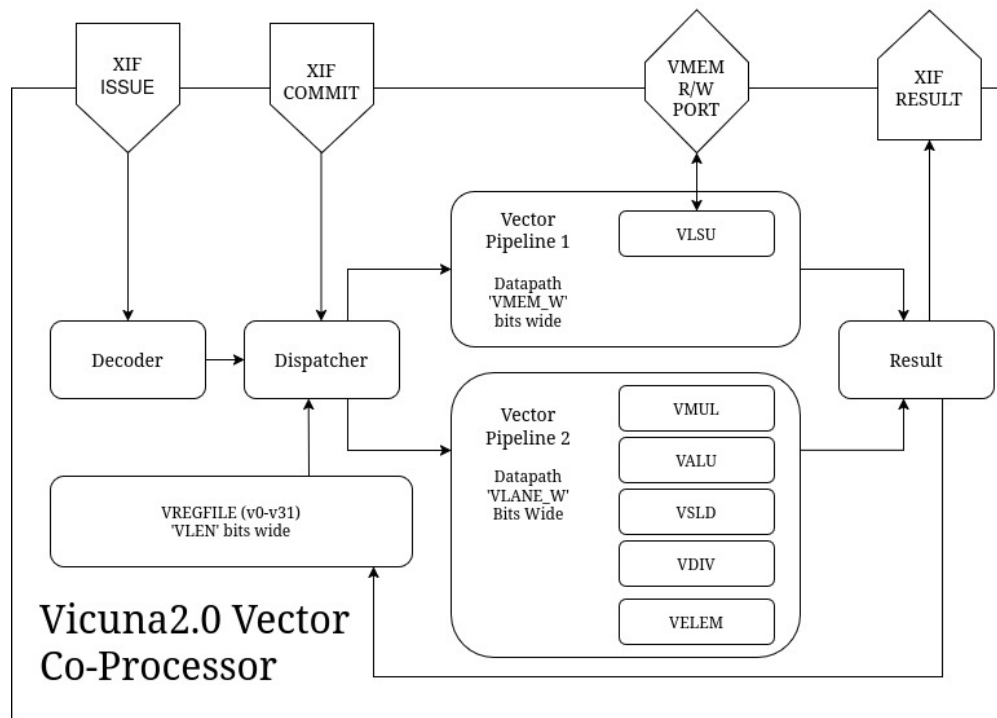
- VLSU
 - Load-Store Unit, handles all memory operations
 - 4 cycle latency, 1 cycle initialization interval
- VMUL
 - Multiply Unit, handles multiplication and MAC
 - 5 cycle latency, 1 cycle initialization interval
- VALU
 - Arithmetic Logic Unit, handles arithmetic and comparisons
 - 3 cycle latency, 1 cycle initialization interval
- VSLD
 - Slide Unit, handles permutation operations
 - variable latency and initialization interval
- VDIV
 - Division Unit, handles division and remainder
 - 34 cycle latency, 32 cycle initialization interval
- VELEM
 - Elementwise Unit, handles
 - variable latency and initialization interval
 - Can only process 1 element per cycle



ALL Functional Units are required to be compliant with '**Zve32x**'

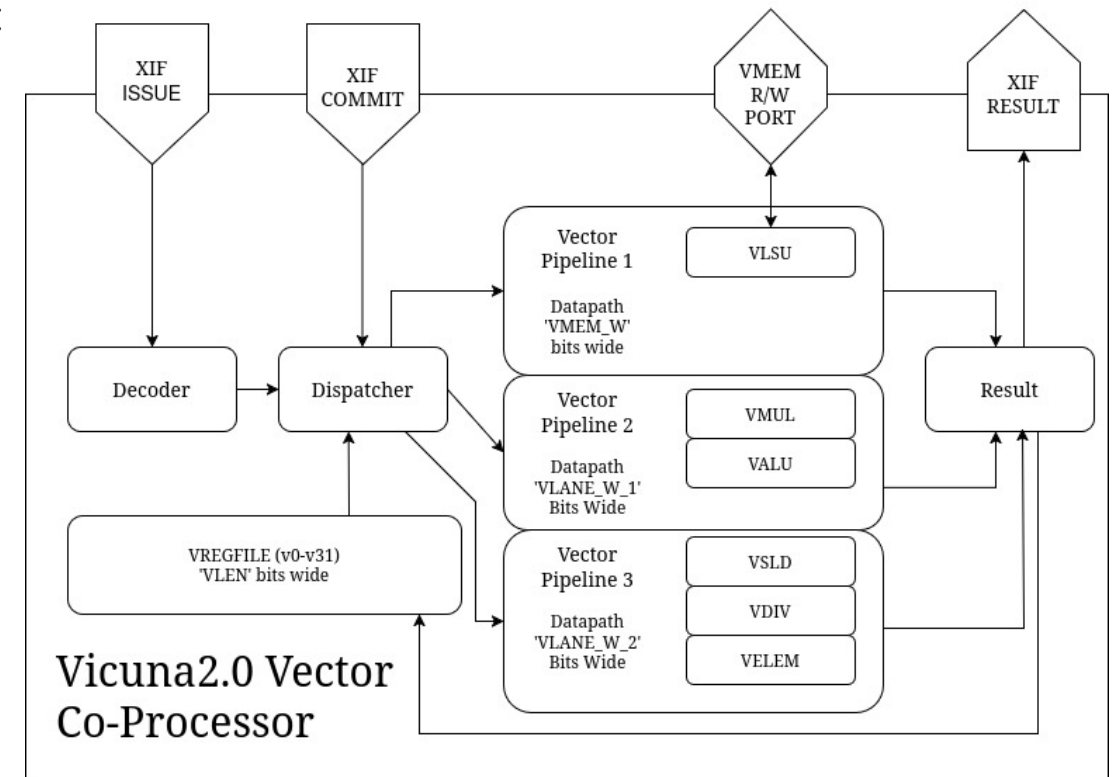
Vicuna2.0 Configuration Parameters

- **Vector Register Width (VLEN bits)**
- **Number of Vector Pipelines**
- **Width of Vector Pipelines (bits) -> Vector Lanes**



Vicuna2.0: Timing Behavior

- Only one vector operation executed per pipeline
 - Even if operations use different functional units
- 3 vector register read ports, 1 write port
 - For reads, entire register is read and pipeline is fed with a shift register until all data consumed
 - For writes, shift register rebuilds output from pipeline and performs write once entire register data is collected
- Chaining is performed through the vector register file
 - Next operation can begin only after an entire vector register has been written
- Scalar core only stalls for:
 - Vector LSU operations
 - Transfer from vector register to scalar register



C Vector Intrinsics

C Vector Intrinsics

- API in C to allow programmers to write RISC-V Vector Code without writing assembly
- Provides vector datatypes and functions
 - Datatypes : `vint8m1_t`, `vint16m2_t`, `vfloat32mf2_t`, etc.
 - Functions: `__riscv_vadd_vv_i16m2`, `__riscv_vadd_vx_i16m2`, `__riscv_vadd_vv_i16m1`, etc.
- Include with `#include "riscv_vector.h"`

```
size_t vl = __riscv_vsetvl_e8m1(len);  
vint8m1_t in1 = __riscv_vle8_v_i8m1(pSrcA, vl);  
vint8m1_t output = __riscv_vadd_vx_i16m2 (in1, scalar ,vl);
```

Vector Intrinsics Example

```
volatile size_t vl = __riscv_vsetvl_e8m1(16);  
vint8m1_t in1 = __riscv_vle8_v_i8m1(pSrcA, vl);  
vint8m1_t in2 = __riscv_vle8_v_i8m1(pSrcB, vl);  
vint8m1_t output = __riscv_vadd_vv_i8m1 (in1, in2, vl);  
__riscv_vse8_v_i8m1 (pDst, output, vl);
```

Documentation and more examples available:

<https://github.com/riscv-non-isa/rvv-intrinsic-doc/tree/main>

Lab1: Task Description

Task Description

- Given an example C program:
 - Write a vectorized version of the program
 - Configure the vector processor for the best performance
 - Evaluate your code AND configuration for total execution time and instructions executed
 - Synthesize your vector unit configuration to evaluate resource utilization
 - Targets for performance and resource utilization will be provided

```
set(VREG_W 128)
set(VMEM_W 32)
set(VPROC_PIPELINES "${VMEM_W}:VLSU 32:VALU,VDIV,VMUL,VSLD,VELEM")
```

LUT Usage : 13062

```
Reference Cycles: 41559
Vector Cycles: 35816
13.818908058422966% improvement

Reference Instr: 29269
Vector Instr: 2862
90.22173630803923% improvement
```

Example: Vector Multiplication

Writing Vector Code: Additional Considerations

- Widening/Narrowing instructions can reduce utilization of the read/write ports
- Only 32 vector registers are available
 - Pay attention to live variables to prevent spill
- Vector Units work best with contiguous accesses
 - Strided accesses are supported, but might have very long latencies
- Division is very inefficient
- To minimize area overheads, reductions are performed in the ELEM unit
 - Long vectors will cause very long latencies

Working on your own

Options for Accessing the Lab Material

- Lab Access:
 - TILab accounts will be generated this weekend and should be available next week (Room 4)
 - Access with your student card (<https://www.tilab.tuwien.ac.at/howto.shtml>)
- Virtual Machine:
 - Using the TILab account, a virtual machine containing all sources and dependencies can be downloaded
- Source Files on gitlab:
 - A TUWEL assignment is open asking for gitlab IDs. We will use this to give you access to the material
- Lab Manual will be uploaded to TUWEL

Warning

Warning: This lab has been established recently. So you might encounter Bugs.
Please report them!!!

2nd Warning

Warning: The VM has been recently created and is giant (119GB uncompressed) and 52 GB compressed!

If you want to use it, test it ASAP!

Decompressing will take ages if you don't find a tool with xz algorithm
(USE LINUX ^^)

LAB source code available at gitlab

Please login once into gitlab.tuwien.ac.at and add your gitlab id in the TUWEL Assignment. So that I can add you to the repository for the source code until **5.11.2025!!!!**

(If you missed it you will get kicked out)

Thank you for your attention!